

## *Chapter 13*

---

# Integration Testing

---

In September 1999, the Mars Climate Orbiter mission failed after successfully traveling 416 million miles in 41 weeks. It disappeared just as it was to begin orbiting Mars. The fault should have been revealed by integration testing: Lockheed Martin Astronautics used acceleration data in English units (pounds), while the Jet Propulsion Laboratory did its calculations with metric units (newtons). NASA announced a \$50,000 project to discover how this could have happened (Fordahl, 1999). They should have read this chapter.

Of the three distinct levels of software testing—unit, integration, and system—integration testing is the least well understood of these; hence in practice, it is the phase most poorly done. This chapter examines two mainline and one less well-known integration testing strategies. They are illustrated with a continuing procedural example, discussed in some detail, and then critiqued with respect to their advantages and disadvantages.

Craftspersons are recognized by two essential characteristics: they have a deep knowledge of the tools of their trade, and they have a similar knowledge of the medium in which they work so that they understand their tools in terms of how they work with the medium. In Chapters 5 through 10, we focused on the tools (techniques) available to the testing craftsperson at the unit level. Our goal there was to understand testing techniques in terms of their advantages and limitations with respect to particular types of software. Here, we continue our emphasis on model-based testing, with the goal of improving the testing craftsperson's judgment through a better understanding of three underlying models. Integration testing for object-oriented software is integrated into this chapter.

### **13.1 Decomposition-Based Integration**

Mainline introductory software engineering texts, for example, Pressman (2005) and Schach (2002), typically present four integration strategies based on the functional decomposition tree of the procedural software: top-down, bottom-up, sandwich, and the vividly named “big bang.” Many classic software testing texts echo this approach, Deutsch (1982), Hetzel (1988), Kaner et al. (1993), and Mosley (1993), to name a few. Each of these strategies (except big bang) describes the order in which units are to be integrated. We can dispense with the big bang

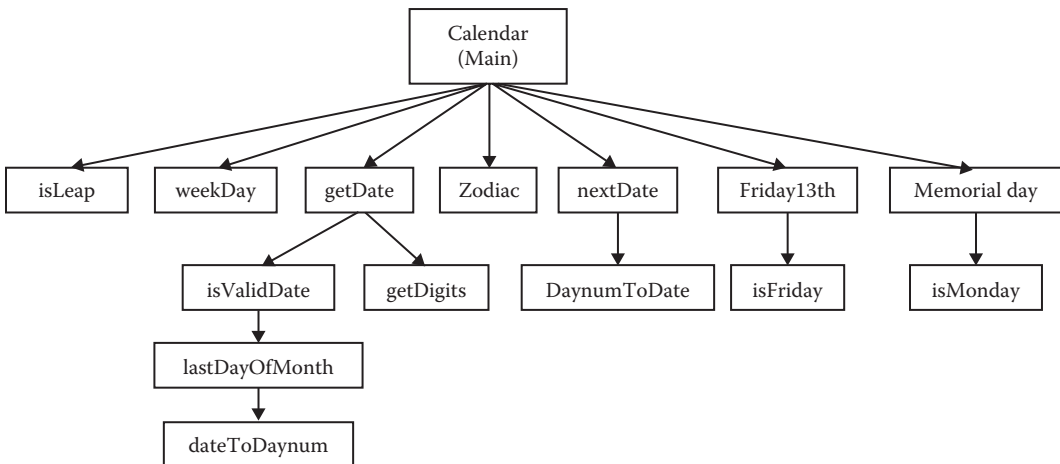
approach most easily: in this view of integration, all the units are compiled together and tested at once. The drawback to this is that when (not if!) a failure is observed, few clues are available to help isolate the location(s) of the fault. (Recall the distinction we made in Chapter 1 between faults and failures.)

The functional decomposition tree is the basis for integration testing because it is the main representation, usually derived from final source code, which shows the structural relationship of the system with respect to its units. All these integration orders presume that the units have been separately tested; thus, the goal of decomposition-based integration is to test the interfaces among separately tested units. A functional decomposition tree reflects the lexicological inclusion of units, in terms of the order in which they need to be compiled, to assure the correct referential scope of variables and unit names. In this chapter, our familiar NextDate unit is extended to a main program, Calendar, with procedures and functions. Figure 13.1 contains the functional decomposition tree for the Calendar program. The pseudocode is given in next.

The Calendar program sketched here in pseudocode acquires a date in the form mm, dd, yyyy, and provides the following functional capabilities:

- The date of the next day (our old friend, NextDate)
- The day of the week corresponding to the date (i.e., Monday, Tuesday, ...)
- The zodiac sign of the date
- The most recent year in which Memorial Day was celebrated on May 27
- The most recent Friday the 13th

A sketch of the Calendar program is given next, followed by a condensed “skeleton,” which is the basis for the functional decomposition in Figure 13.1.



**Figure 13.1** Functional decomposition of Calendar program.

## Pseudocode for the Calendar Program

```

Main    Calendar
Data Declarations
    mm, dd, yyyy, dayNumber, dayName, zodiacSign
Function isLeap (input yyyy, returns T/F)
    (isLeap is self-contained)
End Function isLeap

Procedure getDate (returns mm, dd, yyyy, dayNumber)
    Function isValidDate (inputs mm, dd, yyyy; returns T/F)
        Function lastDayOfMonth (inputs mm, yyyy, returns 28, 29, 30, or 31)
            lastDayOfMonth body
                (uses isLeap)
            end lastDayOfMonth body
        End Function lastDayOfMonth

        isValidDate body
            (uses lastDayOfMonth)
        end isValidDate body
    End Function isValidDate

    Procedure getDigits(returns mm, dd, yyyy)
        (uses Function isValidDate)
    End Procedure getDigits

    Procedure memorialDay (inputs mm, dd, yyyy; returns yyyy)
        Function isMonday (inputs mm, dd, yyyy; returns T/F)
            (uses weekDay)
        End Function isMonday

        memorialDaybody
            isMonday
        end memorialDay
    End Procedure memorialDay

    Procedure friday13th (inputs mm, dd, yyyy; returns mm1, dd1, yyyy1)
        Function isFriday (inputs mm, dd, yyyy; returns T/F)
            (uses weekDay)
        End Function isFriday

        friday13th body
            (uses isFriday)
        end friday13th
    End Procedure friday13th

    getDate body
        getDigits
        isValidDate
        dateToDayNumber
    end getDate body
End Procedure getDate
Procedure nextDate (input daynum, output mm1, dd1, yyyy1)
    Procedure dayNumToDate

```

```

    dayNumToDate body
        (uses isLeap)
    end dayNumToDate body
nextDate body
    dayNumToDate
end nextDate body
End Procedure nextDate

Procedure weekDay (input mm, dd, yyyy; output dayName)
    (uses Zeller's Congruence)
End Procedure weekDay

Procedure zodiac (input dayNumber; output dayName)
    (uses dayNumbers of zodiac cusp dates)
End Procedure zodiac

Main program body
    getDate
    nextDate
    weekDay
    zodiac
    memorialDay
    friday13th
End Main program body

```

### Lexicological Inclusion of the Calendar Program

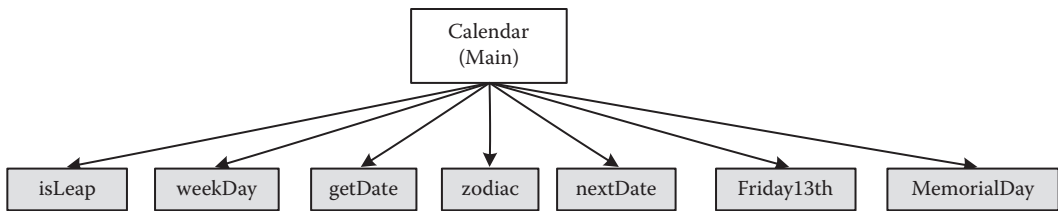
```

Main    Calendar
    Function isLeap
    Procedure weekDay
    Procedure getDate
        Function isValidDate
            Function lastDayOfMonth
        Procedure getDigits
    Procedure memorialDay
        Function isMonday
    Procedure friday13th
        Function isFriday
    Procedure nextDate
        Procedure dayNumToDate
    Procedure zodiac

```

#### ***13.1.1 Top-Down Integration***

Top-down integration begins with the main program (the root of the tree). Any lower-level unit that is called by the main program appears as a “stub,” where stubs are pieces of throwaway code that emulate a called unit. If we performed top-down integration testing for the Calendar program, the first step would be to develop stubs for all the units called by the main program—isLeap, weekDay, getDate, zodiac, nextDate, friday13th, and memorialDay. In a stub for any unit, the tester hard codes in a correct response to the request from the calling/invoking unit. In the stub for zodiac, for example, if the main program calls zodiac with 05, 27, 2012, zodiacStub would return “Gemini.” In extreme practice, the response might be “pretend zodiac returned Gemini.”

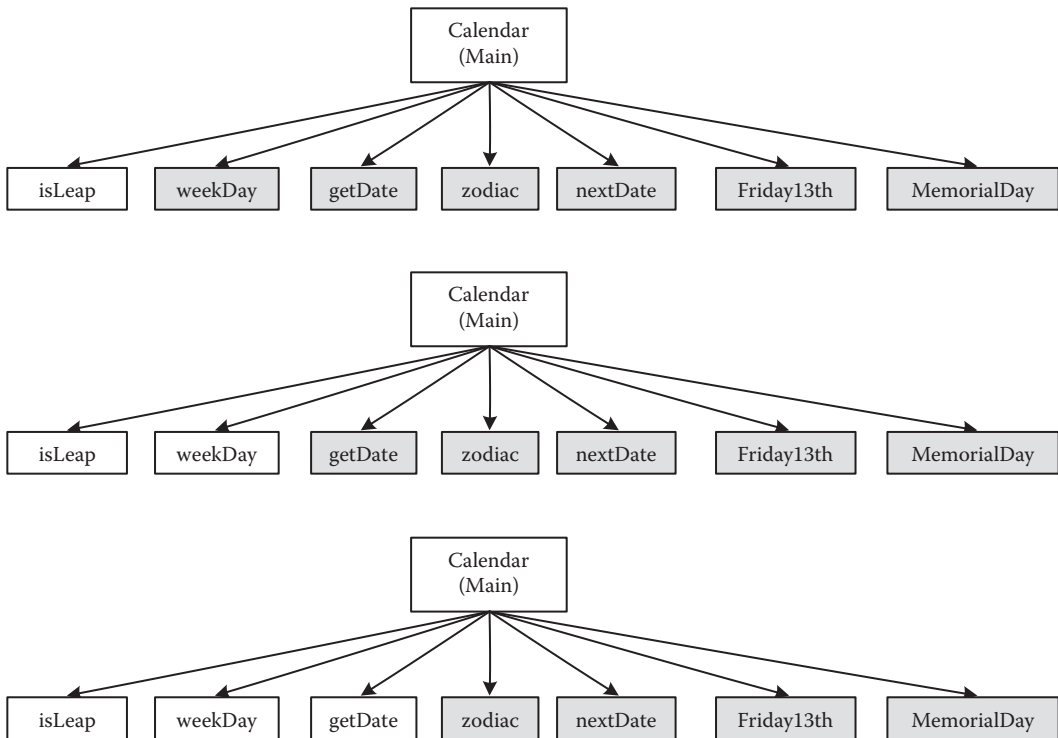


**Figure 13.2** First step in top-down integration.

The use of the pretend prefix emphasizes that it is not a real response. In practice, the effort to develop stubs is usually quite significant. There is good reason to consider stub code as part of the software project and maintain it under configuration management. In Figure 13.2, the first step in top-down integration is shown. The gray-shaded units are all stubs. The goal of the first step is to check that the main program functionality is correct.

Once the main program has been tested, we replace one stub at a time, leaving the others as stubs. Figure 13.3 shows the first three steps in the gradual replacement of stubs by actual code. The stub replacement process proceeds in a breadth-first traversal of the decomposition tree until all the stubs have been replaced. (In Figures 13.2 and 13.3, the units below the first level are not shown because they are not needed.)

The “theory” of top-down integration is that, as stubs are replaced one at a time, if there is a problem, it must be with the interface to the most recently replaced stub. (Note that the fault



**Figure 13.3** Next three steps in top-down integration.

isolation is similar to that of test-driven development.) The problem is that a functional decomposition is deceptive. Because it is derived from the lexicological inclusion required by most compilers, the process generates impossible interfaces. Calendar main never directly refers to either `isLeap` or `weekDay`, so those test sessions could not occur.

### 13.1.2 Bottom-Up Integration

Bottom-up integration is a “mirror image” to the top-down order, with the difference that stubs are replaced by driver modules that emulate units at the next level up in the tree. (In Figure 13.4, the gray units are drivers.) Bottom-up integration begins with the leaves of the decomposition tree, and use a driver version of the unit that would normally call it to provide it with test cases. (Note the similarity to test driver units at the unit level. As units are tested, the drivers are gradually replaced, until the full decomposition tree has been traversed. Less throwaway code exists in bottom-up integration, but the problem of impossible interfaces persists.

Figure 13.5 shows one case where a unit (`zodiac`) can be tested with a driver. In this case, the Calendar driver would probably call `zodiac` with 36 test dates that are the day before a cusp date, the cusp date, and the day after the cusp date. The cusp date for Gemini is May 21, so the driver would call `zodiac` three times, with May 20, May 21, and May 22. The expected responses would be “Taurus,” “Gemini,” and “Gemini,” respectively. Note how similar this is to the assert mechanism in the `jUnit` (and related) test environments.

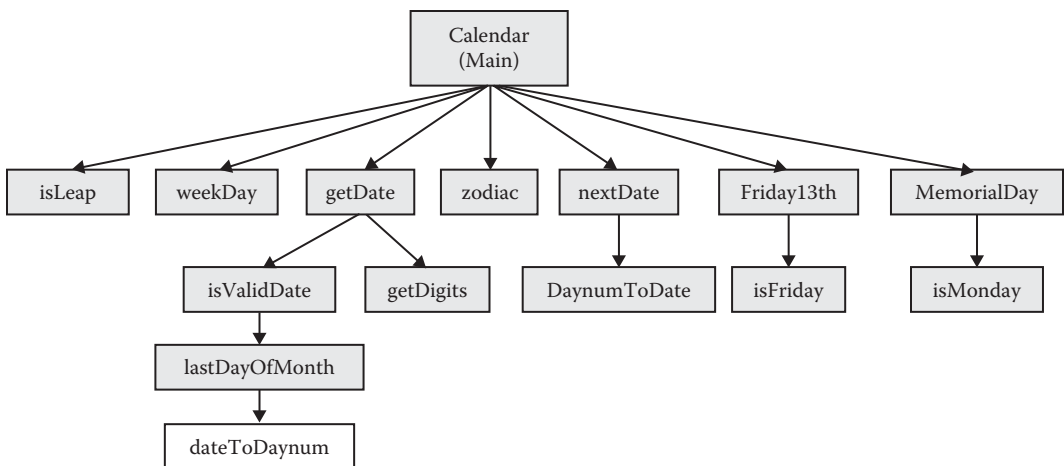


Figure 13.4 First steps in bottom-up integration.

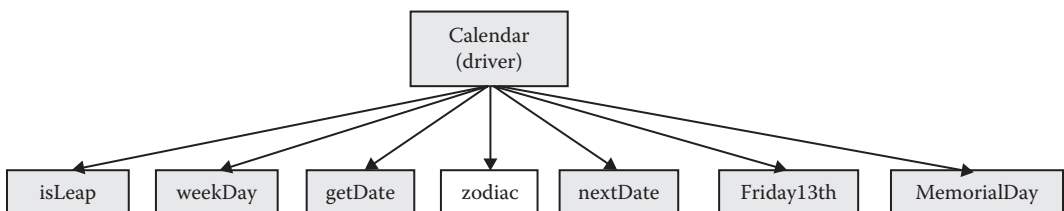


Figure 13.5 Bottom-up integration for `zodiac`.

### 13.1.3 Sandwich Integration

Sandwich integration is a combination of top–down and bottom–up integration. If we think about it in terms of the decomposition tree, we are really only doing big bang integration on a subtree (see Figure 13.6). There will be less stub and driver development effort, but this will be offset to some extent by the added difficulty of fault isolation that is a consequence of big bang integration. (We could probably discuss the size of a sandwich, from dainty finger sandwiches to Dagwood-style sandwiches, but not now.)

A sandwich is a full path from the root to leaves of the functional decomposition tree. In Figure 13.6, the set of units is almost semantically coherent, except that `isLeap` is missing. This set of units could be meaningfully integrated, but test cases at the end of February would not be covered. Also note that the fault isolation capability of the top–down and bottom–up approaches is sacrificed. No stubs nor drivers are needed in sandwich integration.

### 13.1.4 Pros and Cons

With the exception of big bang integration, the decomposition-based approaches are all intuitively clear. Build with tested components. Whenever a failure is observed, the most recently added unit is suspected. Integration testing progress is easily tracked against the decomposition tree. (If the tree is small, it is a nice touch to shade in nodes as they are successfully integrated.) The top–down and bottom–up terms suggest breadth-first traversals of the decomposition tree, but this is not mandatory. (We could use full-height sandwiches to test the tree in a depth-first manner.)

One of the most frequent objections to functional decomposition and waterfall development is that both are artificial, and both serve the needs of project management more than the needs of software developers. This holds true also for decomposition-based testing. The whole mechanism is that units are integrated with respect to structure; this presumes that correct behavior follows from individually correct units and correct interfaces. (Practitioners know better.) The development effort for stubs or drivers is another drawback to these approaches, and this is compounded by the retesting effort.

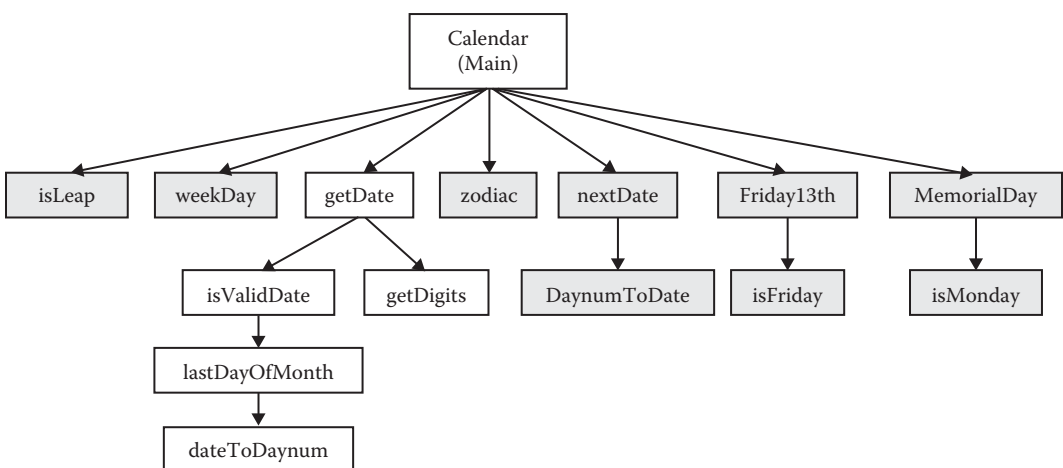


Figure 13.6 Sample sandwich integration.

## 13.2 Call Graph–Based Integration

One of the drawbacks of decomposition-based integration is that the basis is the functional decomposition tree. We saw that this leads to impossible test pairs. If we use the call graph instead, we resolve this deficiency; we also move in the direction of structural testing. The call graph is developed by considering units to be nodes, and if unit A calls (or uses) unit B, there is an edge from node A to node B. The call graph for the Calendar program is shown in Figure 13.7.

Since edges in the call graph refer to actual execution–time connections, the call graph avoids all the problems we saw in the decomposition tree–based versions of integration. In fact, we could repeat the discussion of Section 13.1 based on stubs and drivers in the units in Figure 13.7. This will work well, and it preserves the fault isolation feature of the decomposition-based approaches. Figure 13.8 shows the first step in call graph–based top–down integration.

The stubs in the first session could operate as follows. When the Calendar main program calls `getDateStub`, the stub might return May 27, 2013. The `zodiacStub` would return “Gemini,” and so on. Once the main program logic is tested, the stubs would be replaced, as we discussed in Section 13.1. The three strategies of Section 13.1 will all work well when stubs and drivers are based on the call graph rather than the functional decomposition.

We are in a position to enjoy the investment we made in the discussion of graph theory. Because the call graph is a directed graph, why not use it the way we used program graphs? This leads us to two new approaches to integration testing: we will refer to them as pairwise integration and neighborhood integration.

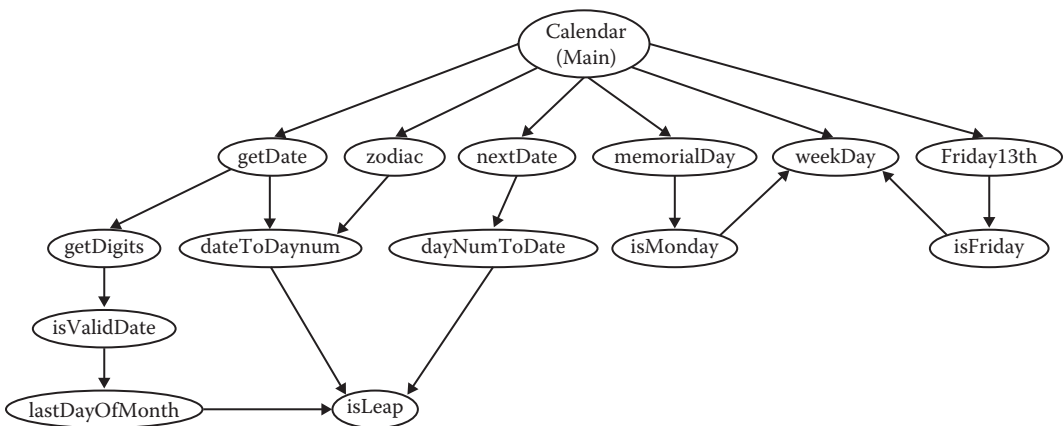


Figure 13.7 Call graph of Calendar program.

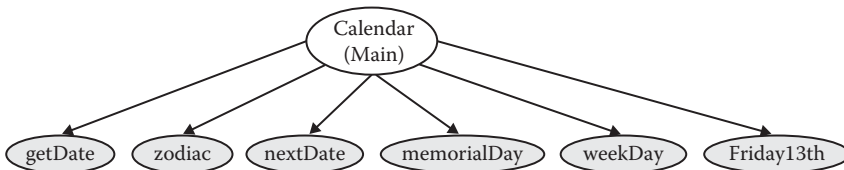


Figure 13.8 Call graph–based top–down integration of Calendar program.



### 13.2.1 Pairwise Integration

The idea behind pairwise integration is to eliminate the stub/driver development effort. Instead of developing stubs and/or drivers, why not use the actual code? At first, this sounds like big bang integration, but we restrict a session to only a pair of units in the call graph. The end result is that we have one integration test session for each edge in the call graph. Pairwise integration results in an increased number of integration sessions when a node (unit) is used by two or more other units. In the Calendar example, there would be 15 separate sessions for top-down integration (one for each stub replacement); this increases to 19 sessions for pairwise integration (one for each edge in the call graph). This is offset by a reduction in stub/driver development. Three pairwise integration sessions are shown in Figure 13.9.

The main advantage of pairwise integration is the high degree of fault isolation. If a test fails, the fault must be in one of the two units. The biggest drawback is that, for units involved on several pairs, a fix that works in one pair may not work in another pair. This is yet another example of the testing pendulum discussed in Chapter 10. Call graph integration is slightly better than the decomposition tree-based approach, but both can be removed from the reality of the code being tested.

### 13.2.2 Neighborhood Integration

We can let the mathematics carry us still further by borrowing the notion of a neighborhood from topology. (This is not too much of a stretch—graph theory is a branch of topology.) The neighborhood of a node in a graph is the set of nodes that are one edge away from the given node. (Technically, this is a neighborhood of radius 1; in larger systems, it makes sense to increase the neighborhood radius.) In a directed graph, this includes all the immediate predecessor nodes and all the immediate successor nodes (notice that these correspond to the set of stubs and drivers of the node). The neighborhoods of `getDate`, `nextDate`, `Friday13th`, and `weekDay` are shown in Figure 13.10.

The 15 neighborhoods for the Calendar example (based on the call graph in Figure 13.7) are listed in Table 13.1. To make the table simpler, the original unit names are replaced by node numbers (in Figure 13.11), where the numbering is generally breadth first. The juxtaposition and connectivity is preserved in Figure 13.11.

The information in Table 13.1 is given in Table 13.2 as the adjacency matrix for the call graph. The column sums show the indegrees of each node, and the row sums show the outdegrees.

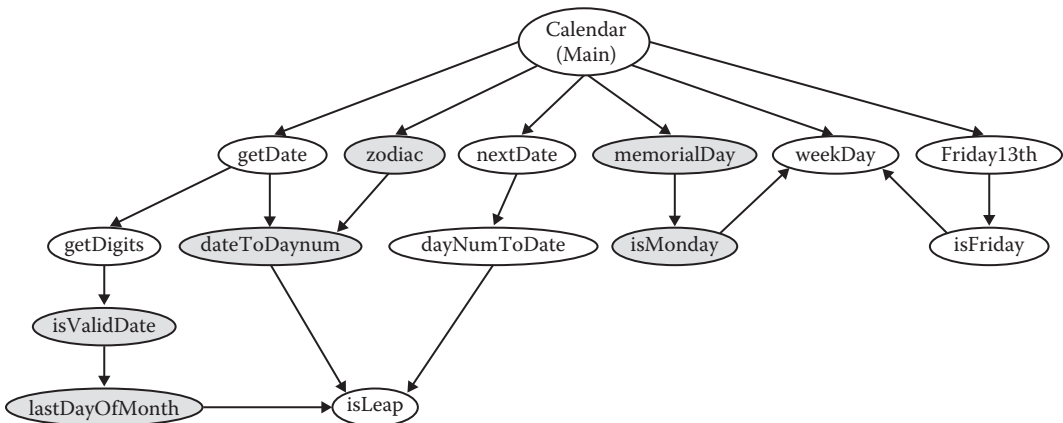
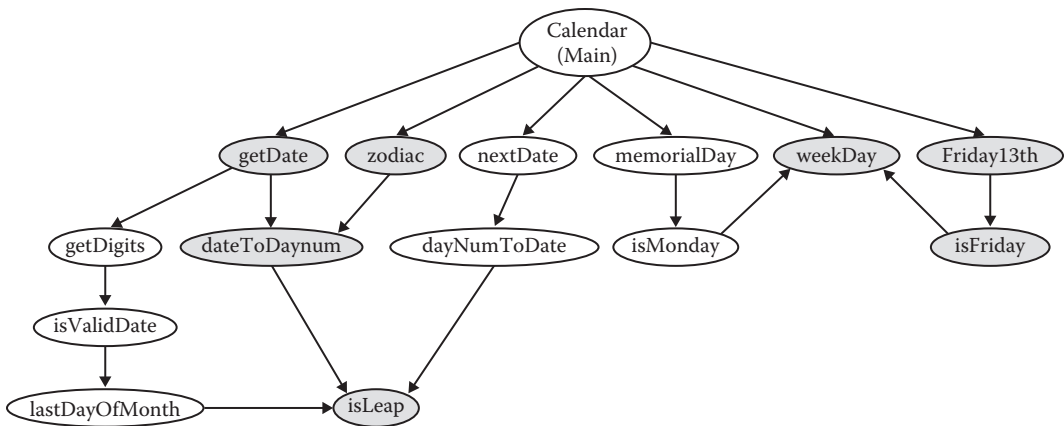


Figure 13.9 Three pairs for pairwise integration.



**Figure 13.10** Three neighborhoods (of radius 1) for neighborhood integration.

**Table 13.1** Neighborhoods of Radius 1 in Calendar Call Graph

| <i>Neighborhoods in Calendar Program Call Graph</i> |                  |                     |                   |
|-----------------------------------------------------|------------------|---------------------|-------------------|
| <i>Node</i>                                         | <i>Unit Name</i> | <i>Predecessors</i> | <i>Successors</i> |
| 1                                                   | Calendar (Main)  | (None)              | 2, 3, 4, 5, 6, 7  |
| 2                                                   | getDate          | 1                   | 8, 9              |
| 3                                                   | zodiac           | 1                   | 9                 |
| 4                                                   | nextDate         | 1                   | 10                |
| 5                                                   | memorialDay      | 1                   | 11                |
| 6                                                   | weekday          | 1, 11, 12           | (None)            |
| 7                                                   | Friday13th       | 1                   | 12                |
| 8                                                   | getDigits        | 2                   | 13                |
| 9                                                   | dateToDayNum     | 3                   | 15                |
| 10                                                  | dayNumToDate     | 4                   | 15                |
| 11                                                  | isMonday         | 5                   | 6                 |
| 12                                                  | isFriday         | 7                   | 6                 |
| 13                                                  | isValidDate      | 8                   | 14                |
| 14                                                  | lastDayOfMonth   | 13                  | 15                |
| 15                                                  | isLeap           | 9, 10, 14           | (None)            |

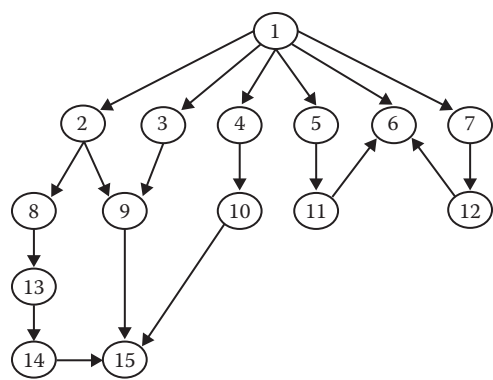


Figure 13.11 Calendar call graph with units replaced by numbers.

Table 13.2 Adjacency Matrix of Calendar Call Graph

|            | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | Row Sum |
|------------|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|---------|
| 1          |   | 1 | 1 | 1 | 1 | 1 | 1 |   |   |    |    |    |    |    |    | 6       |
| 2          |   |   |   |   |   |   |   | 1 | 1 |    |    |    |    |    |    | 2       |
| 3          |   |   |   |   |   |   |   |   | 1 |    |    |    |    |    |    | 1       |
| 4          |   |   |   |   |   |   |   |   |   | 1  |    |    |    |    |    | 1       |
| 5          |   |   |   |   |   |   |   |   |   |    | 1  |    |    |    |    | 1       |
| 6          |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    | 0       |
| 7          |   |   |   |   |   |   |   |   |   |    |    | 1  |    |    |    | 1       |
| 8          |   |   |   |   |   |   |   |   |   |    |    |    | 1  |    |    | 1       |
| 9          |   |   |   |   |   |   |   |   |   |    |    |    |    |    | 1  | 1       |
| 10         |   |   |   |   |   |   |   |   |   |    |    |    |    |    | 1  | 1       |
| 11         |   |   |   |   |   |   | 1 |   |   |    |    |    |    |    |    | 1       |
| 12         |   |   |   |   |   |   | 1 |   |   |    |    |    |    |    |    | 1       |
| 13         |   |   |   |   |   |   |   |   |   |    |    |    |    | 1  |    | 1       |
| 14         |   |   |   |   |   |   |   |   |   |    |    |    |    |    | 1  | 1       |
| 15         |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    | 0       |
| Column sum | 0 | 1 | 1 | 1 | 1 | 1 | 3 | 1 | 2 | 1  | 1  | 1  | 1  | 1  | 3  |         |

We can always compute the number of neighborhoods for a given call graph. Each interior node will have one neighborhood, plus one extra in case leaf nodes are connected directly to the root node. (An interior node has a nonzero indegree and a nonzero outdegree.) We have

$$\text{Interior nodes} = \text{nodes} - (\text{source nodes} + \text{sink nodes})$$

$$\text{Neighborhoods} = \text{interior nodes} + \text{source nodes}$$

which combine to

$$\text{Neighborhoods} = \text{nodes} - \text{sink nodes}$$

Neighborhood integration usually yields a reduction in the number of integration test sessions, and it reduces stub and driver development. The end result is that neighborhoods are essentially the sandwiches that we slipped past in the previous section. (It is slightly different because the base information for neighborhoods is the call graph, not the decomposition tree.) What they share with sandwich integration is more significant—neighborhood integration testing has the fault isolation difficulties of “medium bang” integration.

### 13.2.3 *Pros and Cons*

The call graph–based integration techniques move away from a purely structural basis toward a behavioral basis; thus, the underlying assumption is an improvement. (See the Testing Pendulum in Chapter 10.) The neighborhood-based techniques also reduce the stub/driver development effort. In addition to these advantages, call graph–based integration matches well with developments characterized by builds and composition. For example, sequences of neighborhoods can be used to define builds. Alternatively, we could allow adjacent neighborhoods to merge (into villages?) and provide an orderly, composition-based growth path. All this supports the use of neighborhood-based integration for systems developed by life cycles in which composition dominates.

The biggest drawback to call graph–based integration testing is the fault isolation problem, especially for large neighborhoods. A more subtle but closely related problem occurs. What happens if (when) a fault is found in a node (unit) that appears in several neighborhoods? The adjacency matrix highlights this immediately—nodes with either a high row sum or a high column sum will be in several neighborhoods. Obviously, we resolve the fault in one neighborhood; but this means changing the unit’s code in some way, which in turn means that all the previously tested neighborhoods that contain the changed node need to be retested.

Finally, a fundamental uncertainty exists in any structural form of testing: the presumption that units integrated with respect to structural information will exhibit correct behavior. We know where we are going: we want system-level threads of behavior to be correct. When integration testing based on call graph information is complete, we still have quite a leap to get to system-level threads. We resolve this by changing the basis from call graph information to special forms of paths.

## 13.3 Path-Based Integration

Much of the progress in the development of mathematics comes from an elegant pattern: have a clear idea of where you want to go, and then define the concepts that take you there. We do this here for path-based integration testing, but first we need to motivate the definitions.

We already know that the combination of structural and functional testing is highly desirable at the unit level; it would be nice to have a similar capability for integration (and system) testing. We also know that we want to express system testing in terms of behavioral threads. Lastly, we revise our goal for integration testing: instead of testing interfaces among separately developed and tested units, we focus on interactions among these units. (“Co-functioning” might be a good term.) Interfaces are structural; interaction is behavioral.

When a unit executes, some path of source statements is traversed. Suppose that a call goes to another unit along such a path. At that point, control is passed from the calling unit to the called unit, where some other path of source statements is traversed. We deliberately ignored this situation in Chapter 8, because this is a better place to address the question. Two possibilities are available: abandon the single-entry, single-exit precept and treat such calls as an exit followed by an entry, or suppress the call statement because control eventually returns to the calling unit anyway. The suppression choice works well for unit testing, but it is antithetical to integration testing.

### 13.3.1 New and Extended Concepts

To get where we need to go, we need to refine some of the program graph concepts. As before, these refer to programs written in an imperative language. We allow statement fragments to be a complete statement, and statement fragments are nodes in the program graph.

#### Definition

A *source node* in a program is a statement fragment at which program execution begins or resumes.

The first executable statement in a unit is clearly a source node. Source nodes also occur immediately after nodes that transfer control to other units.

#### Definition

A *sink node* in a unit is a statement fragment at which program execution terminates.

The final executable statement in a program is clearly a sink node; so are statements that transfer control to other units.

#### Definition

A *module execution path* is a sequence of statements that begins with a source node and ends with a sink node, with no intervening sink nodes.

The effect of the definitions thus far is that program graphs now have multiple source and sink nodes. This would greatly increase the complexity of unit testing, but integration testing presumes unit testing is complete.

**Definition**

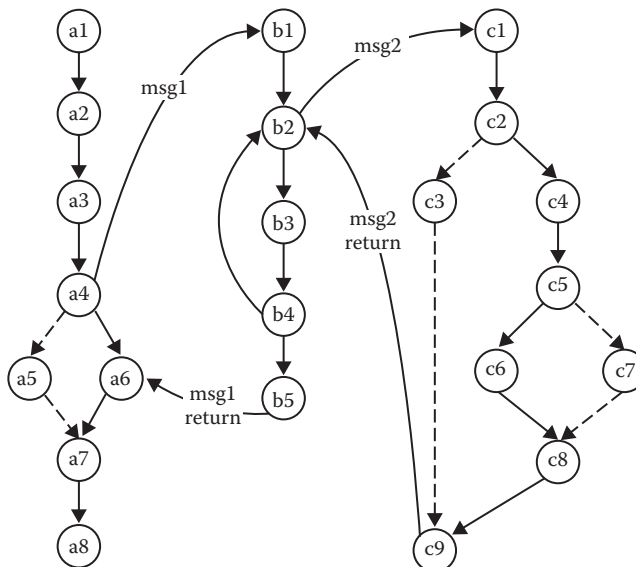
A *message* is a programming language mechanism by which one unit transfers control to another unit, and acquires a response from the other unit.

Depending on the programming language, messages can be interpreted as subroutine invocations, procedure calls, function references, and the usual messages in an object-oriented programming language. We follow the convention that the unit that receives a message (the message destination) always eventually returns control to the message source. Messages can pass data to other units. We can finally make the definitions for path-based integration testing. Our goal is to have an integration testing analog of DD-paths.

**Definition**

An *MM-path* is an interleaved sequence of module execution paths and messages.

The basic idea of an MM-path (Jorgensen 1985; Jorgensen and Erickson 1994) is that we can now describe sequences of module execution paths that include transfers of control among separate units. In traditional software, “MM” is nicely understood as module–message; in object-oriented software, it is clearer to interpret “MM” as method–message. These transfers are by messages, therefore, MM-paths always represent feasible execution paths, and these paths cross unit boundaries. The hypothetical example in Figure 13.12 shows an MM-path (the solid edges) in which module A calls module B, which in turn calls module C. Notice that, for traditional (procedural) software, MM-paths will always begin (and end) in the main program.



**Figure 13.12** Hypothetical MM-path across three units.

In unit A, nodes a1 and a6 are source nodes (a5 and a6 are outcomes of the decision at node a5), and nodes a4 (a decision) and a8 are sink nodes. Similarly, in unit B, nodes b1 and b3 are source nodes, and nodes b2 and b5 are sink nodes. Node b2 is a sink node because control leaves unit B at that point. It could also be a source node, because unit C returns a value used at node b2. Unit C has a single source node, c1, and a single sink node, c9. Unit A contains three module execution paths:  $\langle a1, a2, a3, a4 \rangle$ ,  $\langle a4, a5, a7, a8 \rangle$ , and  $\langle a4, a6, a7, a8 \rangle$ . The solid edges are edges actually traversed in this hypothetical example. The dashed edges are in the program graphs of the units as stand-alone units, but they did not “execute” in the hypothetical MM-path. We can now define an integration testing analog of the DD-path graph that serves unit testing so effectively.

### Definition

Given a set of units, their *MM-path graph* is the directed graph in which nodes are module execution paths and edges correspond to messages and returns from one unit to another.

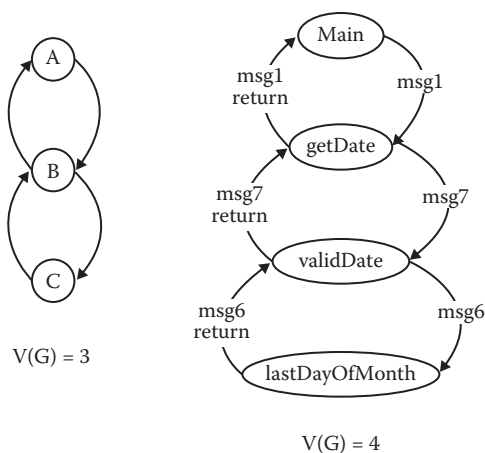
Notice that MM-path graphs are defined with respect to a set of units. This directly supports composition of units and composition-based integration testing. We can even compose down to the level of individual module execution paths, but that is probably more detailed than necessary.

We should consider the relationships among module execution paths, program paths, DD-paths, and MM-paths. A program path is a sequence of DD-paths, and an MM-path is a sequence of module execution paths. Unfortunately, there is no simple relationship between DD-paths and module execution paths. Either might be contained in the other, but more likely, they partially overlap. Because MM-paths implement a function that transcends unit boundaries, we do have one relationship: consider the intersection of an MM-path with a unit. The module execution paths in such an intersection are an analog of a slice with respect to the (MM-path) function. Stated another way, the module execution paths in such an intersection are the restriction of the function to the unit in which they occur.

The MM-path definition needs some practical guidelines. How long (“deep” might be better) is an MM-path? The notion of message quiescence helps here. Message quiescence occurs when a unit that sends no messages is reached (like module C in Figure 13.12). In a sense, this could be taken as a “midpoint” of an MM-path—the remaining execution consists of message returns. This is only mildly helpful. What if there are two points of message quiescence? Maybe a better answer is to take the longer of the two, or, if they are of equal depth, the latter of the two. Points of message quiescence are natural endpoints for an MM-path.

### 13.3.2 MM-Path Complexity

If you compare the MM-paths in Figures 13.12 and 13.17, it seems intuitively clear that the latter is more complex than the former. Because these are strongly connected directed graphs, we can “blindly” compute their cyclomatic complexities; recall the formula is  $V(G) = e - n + 2p$ , where  $p$  is the number of strongly connected regions. Since messages return to the sending unit, we will always have  $p = 1$ , so the formula reduces to  $V(G) = e - n + 2$ . Surprisingly, both graphs have  $V(G) = 7$ . Clearly, MM-path complexity needs some notion of size in addition to cyclomatic complexity (Figure 13.13).



**Figure 13.13** Cyclomatic complexities of two MM-paths.

### 13.3.3 Pros and Cons

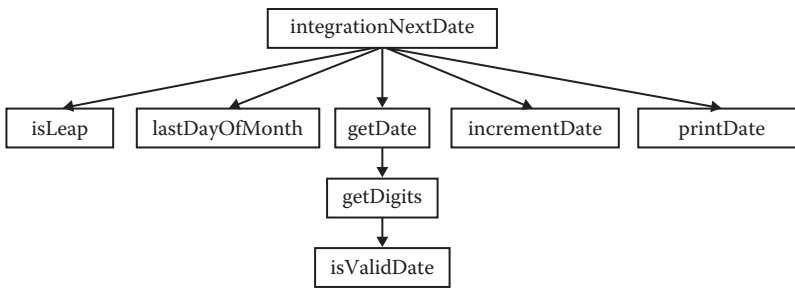
MM-paths are a hybrid of functional and structural testing. They are functional in the sense that they represent actions with inputs and outputs. As such, all the functional testing techniques are potentially applicable. The net result is that the cross-check of the functional and structural approaches is consolidated into the constructs for path-based integration testing. We therefore avoid the pitfall of structural testing, and, at the same time, integration testing gains a fairly seamless junction with system testing. Path-based integration testing works equally well for software developed in the traditional waterfall process or with one of the composition-based alternative life cycle models. Finally, the MM-path concept applies directly to object-oriented software.

The most important advantage of path-based integration testing is that it is closely coupled with actual system behavior, instead of the structural motivations of decomposition and call graph-based integration. However, the advantages of path-based integration come at a price—more effort is needed to identify the MM-paths. This effort is probably offset by the elimination of stub and driver development.

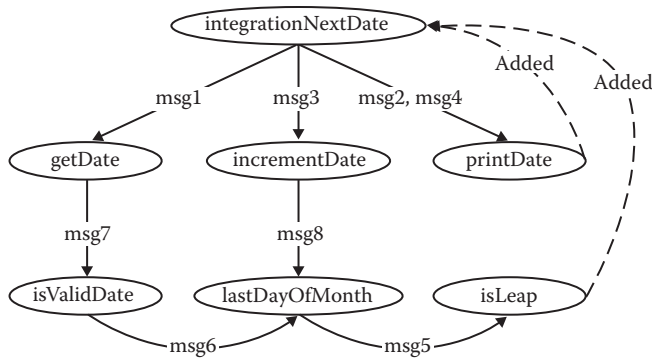
## 13.4 Example: integrationNextDate

Our now familiar NextDate is rewritten here as a main program with a functional decomposition into procedures and functions. This integrationNextDate is a slight extension: there is added validity checking for months, days, and years, so the pseudocode, which follows Figures 13.14 and 13.15, grows from 50 statements to 81. Figures 13.14 and 13.15 show the functional decomposition and the call graph, respectively. Figure 13.16 shows the program graphs of the units in integrationNextDate. Figure 13.17 shows the MM-path for the input date May 27, 2012.





**Figure 13.14** Functional decomposition of integrationNextDate.



**Figure 13.15** Call graph of integrationNextDate.

### 13.4.1 Decomposition-Based Integration

The isLeap and lastDayOfMonth functions are in the first level of decomposition because they must be available to both GetDate and IncrementDate. (We could move isLeap to be contained within the scope of lastDayOfMonth.) Pairwise integration based on the decomposition in Figure 13.14 is problematic; the isLeap and lastDayOfMonth functions are never directly called by the Main program, so these integration sessions would be empty. Bottom-up pairwise integration starting with isLeap, then lastDayOfMonth, ValidDate, and GetDate would be useful. The pairs involving Main and GetDate, IncrementDate, and PrintDate are all useful (but short) sessions. Building stubs for ValidDate and lastDayOfMonth would be easy.

### 13.4.2 Call Graph-Based Integration

Pairwise integration based on the call graph in Figure 13.15 is an improvement over that for the decomposition-based pairwise integration. Obviously, there are no empty integration sessions because edges refer to actual unit references. There is still the problem of stubs. Sandwich integration is appropriate because this example is so small. In fact, it lends itself to a build sequence. Build 1 could contain Main and PrintDate. Build 2 could contain Main, IncrementDate, lastDayOfMonth, and IncrementDate in addition to the already present PrintDate. Finally, build 3 would add the remaining units, GetDate and ValidDate.

Neighborhood integration based on the call graph would likely proceed with the neighborhoods of ValidDate and lastDayOfMonth. Next, we could integrate the neighborhoods of GetDate and IncrementDate. Finally, we would integrate the neighborhood of Main. Notice that these neighborhoods form a build sequence.

#### integrationNextDate pseudocode

```

1  Main integrationNextDate 'start program event occurs here
    Type      Date
              Month As Integer
              Day As Integer
              Year As Integer
    EndType
    Dim today, tomorrow As Date
2  Output("Welcome to NextDate!")
3  GetDate(today)                                'msg1
4  PrintDate(today)                              'msg2
5  tomorrow = IncrementDate(today)              'msg3
6  PrintDate(tomorrow)                          'msg4
7  End Main
8  Function isLeap(year) Boolean
9      If (year divisible by 4)
10         Then
11             If (year is NOT divisible by 100)
12                 Then isLeap = True
13             Else
14                 If (year is divisible by 400)
15                     Then isLeap = True
16                 Else isLeap = False
17             EndIf
18         EndIf
19 Else isLeap = False
20 EndIf
21 End (Function isLeap)

22 Function lastDayOfMonth(month, year) Integer
23     Case month Of
24         Case 1: 1, 3, 5, 7, 8, 10, 12
25             lastDayOfMonth = 31
26         Case 2: 4, 6, 9, 11
27             lastDayOfMonth = 30
28         Case 3: 2
29             If (isLeap(year))                    'msg5
30                 Then lastDayOfMonth = 29
31             Else lastDayOfMonth = 28
32             EndIf
33     EndCase
34 End (Function lastDayOfMonth)
35 Function GetDate(aDate) Date
    dim aDate As Date

36     Function ValidDate(aDate) Boolean 'within scope of GetDate
        dim aDate As Date
        dim dayOK, monthOK, yearOK As Boolean
37         If ((aDate.Month > 0) AND (aDate.Month <=12))

```

```

38         Then      monthOK = True
39                 Output("Month OK")
40         Else      monthOK = False
41                 Output("Month out of range")
42     EndIf
43     If (monthOK)
44         Then
45             If ((aDate.Day > 0) AND (aDate.Day <=
lastDayOfMonth(aDate.Month, aDate.Year))           `msg6
46                 Then      dayOK = True
47                         Output("Day OK")
48                 Else      dayOK = False
49                         Output("Day out of range")
50             EndIf
51         EndIf
52     If ((aDate.Year > 1811) AND (aDate.Year <=2012)
53         Then      yearOK = True
54                 Output("Year OK")
55         Else      yearOK = False
56                 Output("Year out of range")
57     EndIf
58     If (monthOK AND dayOK AND yearOK)
59         Then ValidDate = True
60                 Output("Date OK")
61         Else ValidDate = False
62                 Output("Please enter a valid date")
63     EndIf
64     End (Function ValidDate)

\ GetDate body begins here
65     Do
66         Output("enter a month")
67         Input(aDate.Month)
68         Output("enter a day")
69         Input(aDate.Day)
70         Output("enter a year")
71         Input(aDate.Year)
72         GetDate.Month = aDate.Month
73         GetDate.Day = aDate.Day
74         GetDate.Year = aDate.Year
75     Until (ValidDate(aDate))           `msg7
76     End (Function GetDate)
77     Function IncrementDate(aDate)Date
78         If (aDate.Day < lastDayOfMonth(aDate.Month))   `msg8
79             Then aDate.Day = aDate.Day + 1
80             Else aDate.Day = 1
81                 If (aDate.Month = 12)
82                     Then      aDate.Month = 1
83                             aDate.Year = aDate.Year + 1
84                     Else      aDate.Month = aDate.Month + 1
85                 EndIf
86             EndIf
87     End (IncrementDate)

88 Procedure PrintDate(aDate)
89     Output("Day is ", aDate.Month, "/", aDate.Day, "/", aDate.Year)
90 End (PrintDate)

```

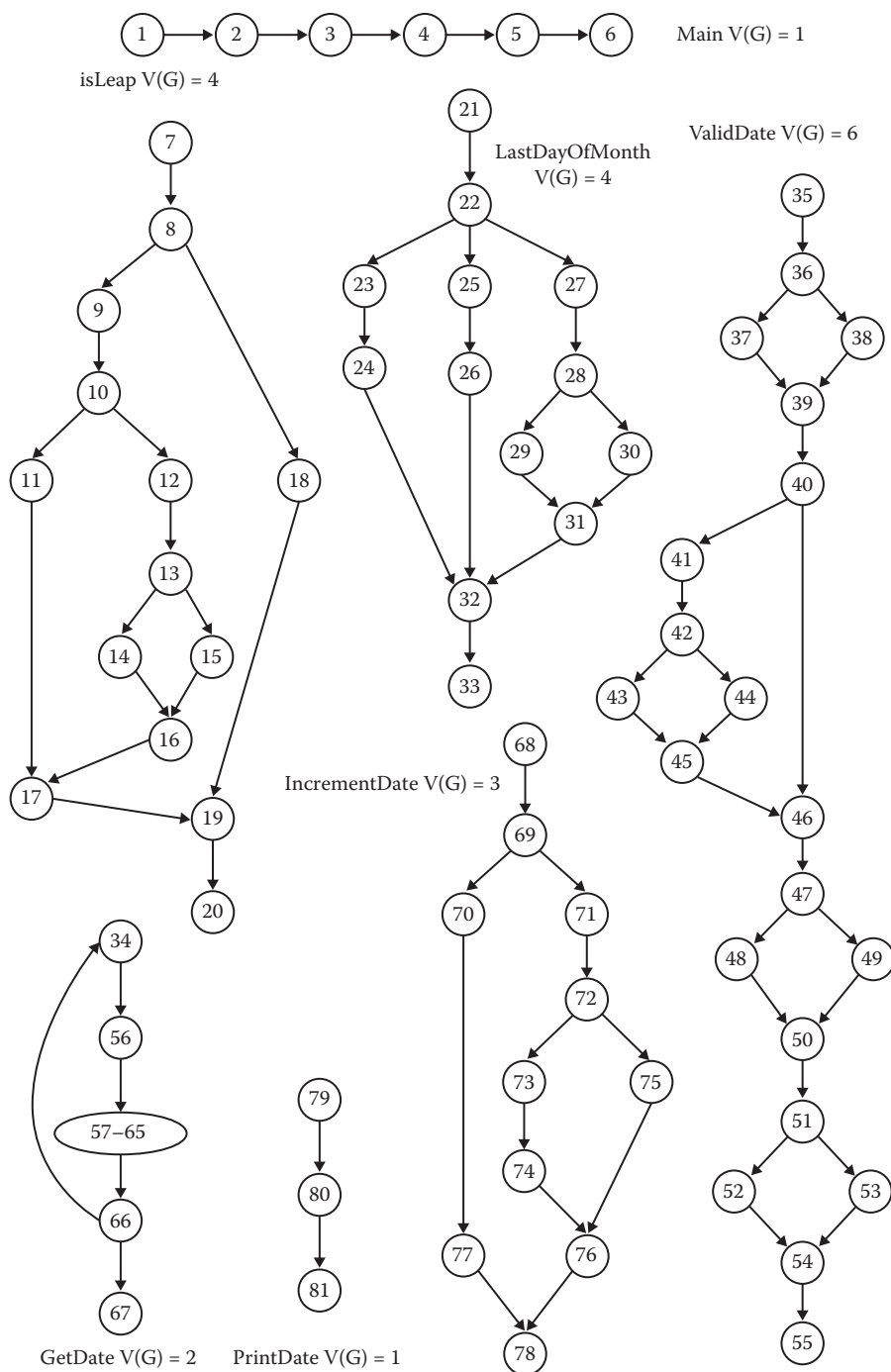


Figure 13.16 Program graphs of units in `integrationNextDate`.

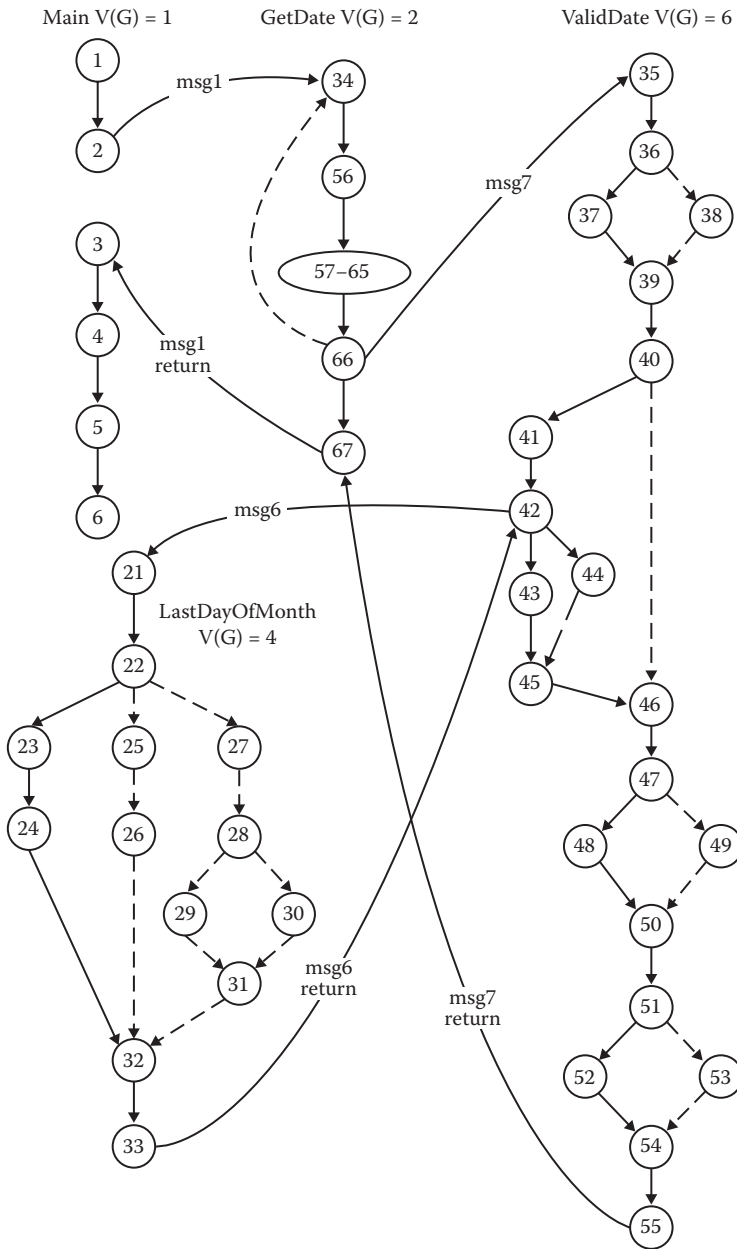


Figure 13.17 MM-path for May 27, 2012.

### 13.4.3 MM-Path-Based Integration

Because the program is data-driven, all MM-paths begin in and return to the main program. Here is the first MM-path for May 27, 2012 (there are others when the Main program calls PrintDate and IncrementDate). It is shown in Figure 13.17.

```

Main (1, 2)
    msg1
    GetDate (34, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66)
        msg7
        validDate (35, 36, 37, 39, 40, 41, 42))
            msg6
            lastDayOfMonth (21, 22, 23, 24, 32, 33)
                'point of message quiescence
            ValidDate (43, 45, 46, 47, 48, 50, 51, 52, 54, 55)
    GetDate (67)
Main (3)

```

We are now in a strong position to describe how many MM-paths are sufficient: the set of MM-paths should cover all source-to-sink paths in the set of units. This is subtly present in Figure 13.17. The solid edges are in the MM-path, but the dashed edges are not. When loops are present, condensation graphs will result in directed acyclic graphs, thereby resolving the problem of potentially infinite (or excessively large) number of paths.

## 13.5 Conclusions and Recommendations

Table 13.3 summarizes the observations made in the preceding discussion. The significant improvement of MM-paths as a basis for integration testing is due to their exact representation of dynamic software behavior. MM-paths are also the basis for present research in data flow (define/use) approaches to integration testing. Integration testing with MM-paths requires extra effort. As a fallback position, perform integration testing based on call graphs.

**Table 13.3 Comparison of Integration Testing Strategies**

| <i>Strategy Basis</i>    | <i>Ability to Test Interfaces</i> | <i>Ability to Test Co-Functionality</i> | <i>Fault Isolation Resolution</i>        |
|--------------------------|-----------------------------------|-----------------------------------------|------------------------------------------|
| Functional decomposition | Acceptable but can be deceptive   | Limited to pairs of units               | Good, to faulty unit                     |
| Call graph               | Acceptable                        | Limited to pairs of units               | Good, to faulty unit                     |
| MM-path                  | Excellent                         | Complete                                | Excellent, to faulty unit execution path |